

# MSME Credit Pipeline Scheduling Assignment

## Task 1

**Question** Prove that the decision version of the MSME Credit Pipeline Scheduling Problem is NP-Hard by reducing from 3-SAT. Define the decision problem, construct the reduction, prove correctness in both directions, include conflict, capacity, SLA constraints and penalty bound, and show the reduction is polynomial-time.

**Answer** This section demonstrates the NP-Hardness of the **MSME Credit Pipeline Scheduling Problem** via a polynomial-time Karp reduction from the NP-Complete problem, **3-SAT**. The idea behind such a reduction is that if we can express any given instance of the 3-SAT problem as an equivalent instance of the scheduling problem in polynomial time, then the 3-SAT problem will be as easily solvable as the scheduling problem. Because 3-SAT is NP-Complete, our scheduling problem is also NP-Hard.

The decision version of the MSME Credit Pipeline Scheduling Problem asks if a given assignment of the scheduling tasks is feasible. The answer will be in the affirmative if the assignment does not violate the conflict (F1), capacity (F2), and SLA (F3) constraints. The assignment will also be feasible if the total penalty incurred from the assignment is less than or equal to a given budget **B**.

To understand the reduction, we will start with an arbitrary instance of 3-SAT with **n** Boolean variables and **m** clauses. Each clause will contain exactly three literals. The goal of the 3-SAT problem is to find a satisfying assignment that satisfies all clauses of the Boolean expression.

In this scheduling instance, each variable corresponds to two pipeline tasks. For each variable ( $x_i$ ), we create two tasks to represent the variable's assignment to **True** ( $(T_i^{\{True\}})$ ) and **False** ( $(T_i^{\{False\}})$ ).

The tasks representing the same variable are linked in the conflict graph. According to the **Conflict Constraint (F1)**, two conflicting tasks cannot occupy the same processing slot. Because of this, the scheduler is forced to pick one of the two tasks. This is similar to the choice of assigning a truth value to a variable in the 3-SAT, where exactly one truth value can be assigned to a given variable.

After defining the variable tasks, one additional task is defined for each clause in the formula. These tasks are called **clause tasks**. Their purpose is to define the satisfaction of the associated clause. A clause task can be scheduled if one of the three literal tasks associated with the clause is selected. If a clause has no selected literals, the clause task cannot be assigned to a processing slot. In this case, the scheduling instance is said to be infeasible. Hence, a complete feasible schedule exists only if each clause has at least one satisfied literal. This shows the infeasibility of the scheduling instance if the Boolean formula is unsatisfiable.

A valid reduction must capture the three families of constraints described in the scheduling problem: the conflict constraints, the resource capacity constraints, and the SLA constraints.

**Conflict Constraint (F1)** is captured by the conflict graph. The tasks ( $(T_i^{\{True\}})$ ) and ( $(T_i^{\{False\}})$ ) for a given variable are connected by a conflict edge. Since conflicting tasks cannot be assigned to the same processing slot, the scheduler must assign one of the two tasks. This guarantees that exactly one truth assignment is assigned to each variable.

**Capacity Constraint (F2)** is captured by the task assignment of a resource vector consisting of CPU, RAM, GPU, and Network. Each processing slot is also assigned fixed limits for the same resources. The limits on the processing slots restrict the combination of tasks that can be assigned to the same

slot. If a conflicting combination of tasks is assigned, at least one of the resource limits will be exceeded. The capacity constraints therefore propagate valid logical task assignments.

**SLA Constraint (F3)** is achieved by setting an execution window  $[l_i, u_i]$  for each task. Within the defined window, each task can be assigned a processing slot. The defined windows constrain the scheduler from moving tasks to circumvent conflicts or capacity violations. Consequently, the conflict, resource, and timing constraints are satisfied by any feasible schedule.

Now we establish the validity of the reduction.

Assume we have a feasible solution for the constructed scheduling instance. Since  $((T_i^{\text{True}}))$  and  $((T_i^{\text{False}}))$  are conflicting tasks, we can only schedule one of them per variable. If  $((T_i^{\text{True}}))$  is scheduled, we assign **True** to that variable. Otherwise, we assign **False**. Since we also scheduled all clause tasks, at least one literal task for that clause has been scheduled. Therefore, that clause has at least one task that is satisfied, meaning that the original 3-SAT formula has at least one satisfied assignment.

Now we look at the opposite direction. If the 3-SAT formula is satisfied, we schedule tasks for either of the selected truth assignments for the corresponding variables. Since only one task is selected per variable, the conflict constraints are satisfied. The selected tasks also satisfy CPU, RAM, GPU, and Network constraints of the processing slots. Since each clause has at least one true literal, all clause tasks can be scheduled within their SLA tasks. Therefore, all feasibility constraints are also satisfied. The reduction yields two tasks for each variable and one task for each clause. This can be done in polynomial time. Thus, since a feasible schedule exists if and only if the original 3-SAT instance is satisfiable, the decision version of the **MSME Credit Pipeline Scheduling Problem** is **NP-Hard**.

*Note: still to revise before submitting.*

Your F1 (conflict) gadget is solid. F2 and F3 are currently asserted narratively, not constructed -- a grader could mark this as "reduces only to coloring" (the rubric's named common failure mode).

Add concrete numbers: give each variable's True/False task pair an identical resource vector sized so a slot can hold at most one of them (F2 now independently enforces the exclusivity F1 already gives you).

Give each clause task a window  $[u, u]$  (a single slot) positioned after every variable task's window, so F3 is forcing real temporal ordering rather than being decorative.

Write 2-3 sentences per constraint showing the reduction would break if that constraint's numbers were removed -- that's what "bidirectional, compound formulation" is actually asking for.

## Task 2

Question Design and justify an extended penalty function  $P(\sigma)$  based on the given base penalty. Explain every term, justify why the extension is useful, prove it is computable in polynomial time, and discuss its practical benefits.

Answer The MSME Credit Pipeline Scheduling Problem has the following formulation of the base penalty function:

$$P_{\text{base}}(\sigma) = \sum_{i=1..n} w(t_i) \times \sigma(t_i)$$

where each  $(t_i)$  is a task,  $(w(t_i))$  is the task's associated priority weight, and  $(\sigma(t_i))$  is the assigned time slot for task  $(t_i)$ . This base penalty function provides a larger penalty for tasks that are scheduled in time slots at the end of the scheduling horizon, thus pushing schedulers to process higher priority loan applications first. This is why urgent credit application requests are given higher scheduling priority compared to priority requests of a lower order. This base function, however, is limited in its focus on task priority, leaving scheduling performance to be inadequate. This can be further extended by including a load imbalance penalty. The new penalty function is defined as:

$$P(\sigma) = [\text{sum over } i=1..n \text{ of } w(t_i) \times \sigma(t_i)] + \lambda \times [\text{sum over } j=1..m \text{ of } (L_j - \bar{L})^2]$$

where (m) is the total number of time slots in a schedule, ( $L_j$ ) is the workload of processing time slot (j), ( $\bar{L}$ ) is the average workload of all time slots, and ( $\lambda$ ) is a positive balancing constant. The average workload is defined as:

$$\bar{L} = (1/m) \times [\text{sum over } j=1..m \text{ of } L_j]$$

This penalty function expresses the imbalance of processing time slots and the utilization of available resources, by ensuring that high priority tasks are processed first, while also ensuring that priority tasks do not result in a workload imbalance.

The first part of the penalty function captures the original scheduling goal. It defines a penalty based on task prioritization and the associated processing slot. The scheduler is incentivized to process more critical loan applications before other less important applications due to the higher penalty for the later scheduling of higher-priority tasks. The application of this scheduling goal minimizes the delays for executing credit processing, the fraud detection tasks, and verification tasks of higher priority.

The load imbalance penalty captures the evenness of the distribution of the workload in the available processing slots. Workloads in processing slots are determined by the tasks that are assigned to the slots. The average workload is determined, and the workloads of the processing slots are compared to the average. In determining the penalty for load imbalance, larger differences are emphasized.

The balancing coefficient defines the difference in focus between the two penalties. A larger balancing coefficient places a higher penalty on load imbalance, shifting focus from task prioritization. This provides the scheduler the flexibility to adjust focus based on the system.

The additional penalty effectively addresses the simultaneous processing of various tasks based on the AI operations of the MSME Credit Pipeline. Failing to address workload distribution results in processing bottlenecks due to inefficient resource utilization.

The proposed penalty function is efficient in that it can be processed and evaluated in polynomial time on any feasible schedule. The actual calculation of the penalty consists only of basic arithmetic operations and only requires a single pass through all scheduled tasks and processing slots in order to calculate it. Therefore, adding the load balancing component to the penalty function does not create any significant additional overhead in terms of computational resources.

The first step in evaluating the penalty function is to calculate the workload assigned to each processing slot. This is done by going through all of the scheduled tasks and adding the processing requirement of each one to the workload of the processing slot assigned to that task. Since all tasks are processed once only will result in an ( $O(n)$ ) time complexity, where ( $n$ ) = total number of tasks.

The next step in calculating the penalty function is to compute the average workload for the slot. The total workload is then divided by the total number of processing slots giving the average workload. Once the average workload has been calculated, the value (deviation) of the workload from the average for each processing slot is computed, and then this value is squared and summed to provide the total imbalance penalty. This step will take ( $O(m)$ ) where ( $m$ ) = total number of processing slots. By summing the time complexity of the first and second step to derive the total complexity of evaluating the penalty function yields ( $O(n + m)$ ) complexity. Since both ( $n$ ) and ( $m$ ) will be polynomial in relation to the size of the input, the proposed penalty function will be evaluated in polynomial time.

## Benefits and Conclusion

By improving the existing penalty function, we have achieved increased efficiency in both scheduling and resource usage within the MSME Credit Pipeline. This has been accomplished by incorporating a priority level for tasks that combines the individual task's workload (balance between completed and still needed work) with the scheduler's priority level for the task. In turn, this provides the scheduler

an avenue for placing priority tasks near the front of their execution order, while ensuring the computer resources are evenly utilized throughout the entire execution period. Without this consideration, some processing slots will become overloaded, while other processing slots will remain underutilized.

In addition, the balanced distribution of the workload across the entire system will decrease the average wait times of each task being processed, due to reduced long queues forming on any individual resource pool. Therefore, by processing a greater number of loans within the same amount of time (increased system throughput) means that the overall time taken to complete the entire process is also reduced (e.g., faster processing of loans ultimately results in better service quality through quicker credit decisions and elimination of delays in loan approvals).

The existing penalty function is no simple task. One task's assignment change affects both its priority-based penalty and load imbalance penalty; therefore, the scheduler must optimize multiple objectives being adjusted simultaneously instead of just task priority. Consequently, the scheduler is able to make practical scheduling decisions that reflect operational requirements in real-time.

In summary, the extended penalty function combines priority based scheduling, with balanced allocation of resources resulting in improved scalability, greater utilization of hardware, reduced occurrence of processing bottlenecks, and improved efficiency of large scale MSMEs.

## Task 3: Resource-Aware Priority Load Balancing Scheduler (RPLBS)

Algorithm Name: Resource-Aware Priority Load Balancing Scheduler (RPLBS)

Resource-Aware Priority Load Balancing Scheduler (RPLBS) is the name given to this proposed algorithm because it takes into consideration the priority of tasks, availability of resources, adherence to SLAs, handling conflicts, and balancing workloads into one scheduling approach.

The objective of RPLBS is to provide a feasible schedule for all MSME Credit Pipeline tasks while minimizing the overall scheduling penalty number that was established in Task 2.

RPLBS utilizes a greedy heuristic approach to schedule tasks based on their priority using a descending order of task priority. High priority tasks will be scheduled first and then low priority tasks will be scheduled after, thus minimizing penalties based on priority.

For each individual task, RPLBS inspects each allocated processing time and slot to see if this assignment satisfies all possible constraints for that task's scheduling/processing, including resource capacity limits (CPU, RAM, GPU and Network), conflict restrictions between the task and current tasks that are incompatible, and the task's allowed SLA execution timeframe.

If there are multiple processing times and slots to choose from, RPLBS selects the processing time with the least overall penalty increase based on the original priority penalty plus any additional load imbalance penalty created in Task 2. The use of even distribution of workloads across all processing slots improves resource use and efficiency.

If at any time there are no available processing slots for scheduling a task, the scheduling instance will be determined as infeasible.

### Scheduling Process and Assignment Steps

When an assignment is made, the algorithm shall perform the following steps:

Assign the current work level; Assign the current processing slot and task; Update any work levels. After all viable assignment has been made, you will obtain an order of all assignments (assignment #, process slot, task). If there are any remaining assignments and/ or no viable assignment made, the output is "Infeasible".

## Operation of RPLBS

The RPLBS employs a greedy heuristic strategy that schedules tasks according to their priority while also ensuring that all scheduling constraints are satisfied. First, the algorithm sorts all tasks in priority weight descending order to ensure that urgent MSME loan application processes or critical processing tasks are scheduled first.

For each task, the algorithm scans all available processing slots in the system. The first step is to determine if the task will be able to be scheduled in its designated SLA window. If it can be scheduled, it will be determined if the task and the designated slot create a scheduling conflict with another task that has already been scheduled into that slot. If the task and the designated slot are in agreement with each other's scheduling conflicts, the algorithm then verifies that the task's requirements for CPU, RAM, GPU, and Network will fit within the available resource capacities of that slot.

During this process, if multiple scheduling slots satisfy the previous scheduling criteria, then the algorithm calculates the total penalty associated with each scheduling slot. The total penalty consists of both the penalty associated with scheduling priorities and the load imbalance penalty assessed in Task 2. The algorithm will select the scheduling slot with the lowest total penalty as the best option for that task.

## Scheduling Constraints

Resource-Aware Priority Load Balancer Scheduler (RPLBS) is provided to assure that all scheduling limitations are satisfied while creating a viable schedule. Prior to assigning any task to a particular processing slot, a sequence of validated checks are executed by the algorithm to ensure that every assignment was made in compliance with the system requirements.

The first verification step of the algorithm will evaluate the conflict limitations. Each task assigned to a processing slot is evaluated against all prior assigned tasks. If two tasks have been deemed to be in conflict, they will never be scheduled to run within the same processing slot.

Next, the algorithm will evaluate the resource capacity limitations. The assignment of the current task will only be accepted if the total CPU, RAM, GPU, and Network will remain within the maximum capacity of the processing slot following the assignment of the current task. Finally, the algorithm will enforce the SLA window limitation. Each task has a defined execution window and the RPLBS will assign that task only if it is able to perform the assigned task within the SLA window established by the task.

## Penalty Minimization & Infeasibility Detection

The Resource-Aware Priority Load Balancing Scheduler (RPLBS) reduces penalties incurred when tasks are scheduled by determining both original task precedence-based penalties and additional processing slot penalties created by migration from one processing slot to another.

The algorithm considers how much the overall total penalty will increase for each processing slot if the current task was assigned to that slot, and it then compares all feasible processing slots to determine which feasible processing slot has the lowest overall total penalty.

If a task cannot be scheduled due to SLA window violation, not enough CPU, RAM, GPU, Network resources, or conflicts, it immediately terminates and identifies the scheduling instance as infeasible.

## Time Complexity Analysis / Design Justification

The Resource-Aware Priority Load Balancing Scheduler (RPLBS) is designed to run in polynomial time. The first step of the algorithm is to sort all tasks by their priority weights; this takes  $O(n \log n)$  time. For each task, the scheduler examines all processing slots available; this includes SLA verification, conflict checks, resource validation and penalty calculations. The scheduling portion takes  $O(n \times m)$  time.

Thus, the overall time complexity is  $O(n \log n + n \times m)$ .

Tasks are processed in priority order because delaying high-priority loan applications will increase the overall penalty and impact service quality. The scheduling strategy selects the processing slot that produces the least additional penalty, ensuring efficient resource use. Conflict checks ensure incompatible applications are not scheduled together.

#### Alternative Scheduling Algorithms and Conclusion

The first alternative was First-Come, First-Served (FCFS), which ignores priority and resource constraints and leads to bottlenecks.

The second alternative was Round Robin Scheduling, which distributes tasks fairly but ignores priorities and SLA deadlines. In comparison, RPLBS considers priority, conflicts, resource capacity, SLAs, and load balancing penalties.

It can be concluded that the RPLBS algorithm meets all requirements for scalability in large-scale scheduling environments and provides improvements over traditional scheduling techniques.

To conclude, the RPLBS algorithm provides a highly flexible and effective scheduling methodology that results in completion of jobs in a timely manner with minimal resource consumption, while also providing optimal load balancing across all processing slots.

*Note: still to revise before submitting.*

The spec's deliverable is "structured pseudocode with line-level justification" -- your description above is prose only. No pseudocode currently appears anywhere in the submission.

Convert the algorithm you already described into a numbered Algorithm block (template below). This is a formatting pass on content you already wrote -- fill in the exact conditions from your own Scheduler.java so the pseudocode matches your code line-for-line, since Task 8 asks you to trace this from memory on a fresh instance.

```
Algorithm RPLBS(tasks, slots, K)
Input:  tasks (each with priority w, resource vector r, window [lo,hi],
conflicts)
        slots (each with capacity vector C)
Output: assignment sigma: task -> slot, OR "infeasible"

1:  sort tasks in descending order of priority weight w
2:  for each task t in sorted order:
3:      bestSlot <- null; bestPenalty <- infinity
4:      for each slot s in [lo_t, hi_t]:                      // F3
5:          if exists assigned task t' in s with conflict(t, t'): continue
// F1
6:          if usage(s) + r(t) > C(s) in any dimension: continue
// F2
7:          p <- PenaltyCalculator(t, s)                      // priority term + load-
imbalance term
8:          if p < bestPenalty: bestSlot <- s; bestPenalty <- p
9:          if bestSlot = null:
10:             report INFEASIBLE, violation_reason <- "no feasible slot for " +
t
11:             halt
12:     assign t to bestSlot; update usage(bestSlot)
13: return sigma

-- replace/annotate each line above with why it's there (line-level
justification is graded), and keep this in sync with Scheduler.java.
```

## Task 4: Proof of Correctness and Performance of RPLBS

There are three constraints that must be checked in order for a task to be assigned to a specific slot. These constraints are: 1) Conflict Constraint (F1) - This is to check if there is any conflict with another already scheduled task; 2) Capacity Constraint (F2) - This is to ensure that adding a new task does not result in any capacity violations for CPU, RAM, GPU or Network usage; 3) SLA Constraint (F3) - This is to ensure that the task is completed within the defined SLA window.

Once all three constraints have been satisfied, the task can be assigned to the slot. If no processing slots meet all three constraints, then the algorithm will report that the request is infeasible instead of forcing the assignment of the task. This allows RPLBS to create valid schedules, as every assignment made will adhere to the above constraints.

The correctness of the Resource-Aware Load Balancing Scheduler (RPLBS) is formally established by demonstrating that, at every assignment step, it enforces constraints F1, F2 and F3.

**Conflict Constraint (F1).** The algorithm tracks the active assignment of tasks to processing slots, and checks for conflicting tasks that have been assigned to a slot before it assigns a new task. If a conflicting task has been assigned to the slot, that slot is immediately disqualified for receiving the new task. Thus, two conflicting tasks will never have an assignment in the same processing slot.

**Capacity Constraint (F2).** The algorithm tracks the cumulative resource usage of each processing slot, and before assigning a task to a processing slot, it adds the resource demand for the task to the current load of the processing slot. The assignment of the new task to that processing slot will only be accepted if the resource usage of that slot does not exceed the maximum resources allowable for that processing slot. If it exceeds the allowable utilization, that slot will not be used to assign the task.

**SLA Constraint (F3).** Before assigning a task to a processing slot, the algorithm checks the task's allowed execution time against the current utilization of the processing slot and the possible assignments of the task to that processing slot. If assigning the task to the processing slot results in violating the task's SLA deadline or SLA start-time constraint, the processing slot will be disqualified from receiving the task.

Since all assignments are made only after satisfying F1, F2, and F3, the algorithm always produces valid schedules whenever a feasible assignment exists.

The Resource-Aware Priority Load Balancing Scheduler (RPLBS) uses a heuristic approach to solving the MSME Credit Pipeline Scheduling Problem by minimizing the penalty function defined in Task 2. The penalty function has two components: priority scheduling penalty and load imbalance penalty. The algorithm selects a feasible processing slot at each step with the minimum incremental increase of the total penalty.

Let  $P_{\sigma\_Algorithm}()$  be the definitely incurred by RPLBS (the algorithm) and  $P_{\sigma\_Optimal}()$  be the minimum penalty that could be obtained from an optimal scheduler (the optimal in this context). Since the RPLBS uses a greedy strategy for selecting processing locations, the RPLBS algorithm does not guarantee global optimality, only local optimality by selecting the best feasible location for each choice set.

Therefore, we derive the following performance bounds for the algorithm:

$$P_{\sigma\_Algorithm}() \leq \alpha P_{\sigma\_Optimal}()$$

where  $\alpha \geq 1$  is the approximation factor due to greedy decision making and local optimality.

The value of  $\alpha$  is dependent on task distribution, resources constraints, and the tightness of SLAs. In scenarios with a balanced workload,  $\alpha$  will approach 1 and therefore will provide performance very close to optimal. In scenarios that have a lot of constraint or conflicting tasks,  $\alpha$  will increase because of a limited availability of feasible processing locations. Therefore, the RPLBS algorithm is a bounded approximation guarantee.

To understand RPLBS' worst case performance, consider the scheduling example of using highly constrained resources and tasks with high levels of conflict. Assume three slots S1, S2, and S3. Each has limited CPU, RAM, GPU, and Network capacities. Assume five tasks T1- T5 exist in this scenario that all have very high resource demands and require the same timeframes (SLA) to complete, creating a very high level of conflict. As well, assume most of these 5 tasks are in conflict with each other, except for a very small number of pairs of tasks.

Every task is given a descending priority weight, so all 5 tasks must be placed in slot S1, S2, or S3 based upon the absence of another feasible slot for their assignment following the application of conflict constraints. In this case, RPLBS would operate almost deterministically and there would be very few options available for the slot-based assignment of any task. As each of the five tasks is examined for assignment, each would be assigned to the only feasible slot available after conflict checks.

However, in this case there is a better global method of assignment by re-ordering all task assignments and using the available flexibility of future time windows to minimize penalties associated with the load distribution. RPLBS makes greedy assignments by making early, irrevocable decisions without backtracking, therefore resulting in an accumulation of growing penalties due to the ineffective distribution of loads.

Thus, RPLBS produces a very poor pairwise penalty when compared with the optimal distribution of tasks due to its greedy nature producing locally optimal but globally sub-optimal solutions. The penalty produced by RPLBS is much larger than the optimal penalty, thereby increasing the approximation factor.

*Note: still to revise before submitting.*

The Feasibility Guarantee argument above is reasonable. The Approximation Ratio and Tight Adversarial Example are currently qualitative only -- no formula for alpha, no instance with real numbers. Below is a REAL adversarial instance, found by actually running your fixed implementation against the assignment's own generator (n=8, K=3, density=0.3, seed=1 -- the first required Task 6 instance, not invented).

Use this as your worked example, then write the derivation and interpretation in your own words -- you'll be asked to defend this reasoning live.

```
Instance (generate_instance(n=8, K=3, density=0.3, seed=1)):
  windows: T4=[1,2] T5=[0,2] T6=[0,2] T7=[1,2]
  conflicts: T7 conflicts with T4, T5, AND T6.
             T4, T5, T6 do NOT conflict with each other.
  resources (CPU,RAM,GPU,Net): T4=[7.49,30.16,1.94,1.42]
T5=[8.47,28.48,1.30,1.59]
T6=[8.94,35.69,1.51,1.59] slot capacity (each slot) =
[32,128,8,6.0]
```

```
RPLBS trace (priority order puts T4, T5, T6, T7 early):
  T4 -> slot 1 (empty slot minimizes load-balance penalty)
  T5 -> slot 0 (empty slot minimizes load-balance penalty)
  T6 -> slot 2 (empty slot minimizes load-balance penalty)
  T7 -> window is {1,2}; slot 1 blocked (conflicts T4), slot 2 blocked
(conflicts T6)
=> RPLBS reports INFEASIBLE
```

```
Optimal (found by brute force): pack T4+T5+T6 into slot 2 together
(they don't conflict each other, and 24.9/32 CPU, 94.3/128 RAM, 4.75/8
GPU,
4.6/6.0 Net all fit in ONE slot) -- this frees slot 1 for T7.
Optimal total penalty ~= 6.13 (feasible).
```



Result: RPLBS achieves 0% of optimal on this instance (reports infeasible when a feasible schedule exists) -- so no finite multiplicative ratio  $\alpha$  holds for RPLBS in general, as currently specified. This is caused specifically by your Task 2 load-imbalance term rewarding spread of mutually-compatible tasks across slots, which destroys packing options a later conflicting task needed.

## Task 5: Java Implementation of RPLBS

The Resource-Aware Priority Load Balancing Scheduler (RPLBS) is implemented in Java 17 using only Java standard libraries so that it complies with the requirement that no external optimization or SAT solving tools will be used. The RPLBS program reads a scheduling instance from a JSON file, using the previously-defined greedy heuristic from Task 3 to process the tasks, and produces a structured JSON output.

The input JSON contains a list of tasks, each with a unique task ID, a priority weight, the required resources (CPU, RAM, GPU, Network) and the SLA window constraints. The input JSON also contains a conflict graph with incompatible task pairs and the processing slots, which represent the available resources. Finally, the input JSON provides the number of processing slots, which initializes the RPLBS scheduler.

When the RPLBS program starts, it parses the input JSON into internal Java objects (Task, Slot, SchedulerState). The program uses either a custom JSON parser or org.json-style utilities (if supported by the standard libraries) to convert raw input into the structured data. Each Task object is created with its constraints; each Slot is created with its current resource consumption and its assigned tasks. The structured representation of tasks and slots allows for quick access to tasks and slots during the scheduling process while ensuring that constraint checking (F1, F2, F3) is possible.

Implementing the Resource-Aware Priority Load Balancing Scheduler (RPLBS) Scheduling Algorithm is straightforward, as it uses the same greedy heuristic as described in Task 3. After parsing the input specifications for all of the scheduled tasks, the tasks are sorted in order of high to low by priority weight so that the highest priority MSME credit use schedules first, which allows for critical tasks to execute earlier and decrease penalties associated with priority-based scheduling.

For each task, the algorithm checks all available processing slots to determine whether or not the task can be executed on that slot. A slot's capacity must meet all constraints (conflict constraint (F1), capacity constraint (F2) and SLA window constraint (F3), before any checking of that slot's potential conflict occurs. Each slot is checked for conflict against a list or set of already assigned tasks. If there is conflict with an assigned task, that slot will be rejected immediately. The remaining capacity of each slot must also be checked for resources (CPU, RAM, GPU and Network) to determine if the addition of the new task would exceed the remaining total resource capacity of the slot. The scheduling of the new task must meet the conditions for when the task is allowed to execute (SLA validation). If there are multiple slots that a new task is valid to be executed in, the penalties incurred by each of those slots are calculated based on the penalty function developed in Task 2 by factoring in the penalty due to priority and the penalty due to load imbalance. The slot with the minimum total penalty is selected for that new task.

The penalty function from Task 2 is included in this implementation using a direct calculation approach. Each scheduling decision is evaluated using two different penalty calculations: a priority penalty based on the assigned index of the processing slot (defined by priority weight) and a load imbalance penalty based upon the variance of processing load in the assigned processing slots. The priority penalty calculation bases priority weight and processing slot on the assigned index of the task. The later the index for higher priority tasks, the greater the penalty will be, thus motivating the scheduling of critical MSME credit tasks as early as possible. The load imbalance penalty is determined by computing a standard deviation from the average workload of all processing slots, with the variance of the workload changing with each assignment. The square of the standard deviations

from the average workload will result in higher penalties for unevenly distributed loads versus evenly distributed loads.

The total penalty for a schedule is determined by summing the penalties for each priority and load imbalance penalty across all the tasks in all the processing slots. The final value will be stored in the output JSON under the penalty field.

The following information will be included in the output JSON file: assignment, penalty, runtime\_ms, feasible, violation\_reason. Runtime performance is recorded using System.nanoTime() or System.currentTimeMillis() at the beginning and end of execution, and stored in runtime\_ms.

The implementation of RPLBS incorporates stringent infeasibility detection and mechanism enforcement. If a task cannot be scheduled into any workable slot because it conflicts with F1, F2, or F3 constraints, the system will abort and mark the scheduling problem as infeasible. This prevents partial or incorrect schedules. The output JSON will include a "violation\_reason" field such as "exceeded resource capacity" or "SLA window violation."

The RPLBS algorithm also includes unit tests: fully conflicting tasks test for F1 validation, zero capacity slot test for F2 validation, tight SLA window test for F3 validation, and single task test for basic correctness.

The RPLBS algorithm written in Java has been developed in a modular format, and each modular component implements a section of the RPLBS algorithm according to the design requirements in Task 3. The following are the primary modular components: Task, ProcessingSlot, Scheduler, PenaltyCalculator, InputParser, OutputGenerator. The Task class contains the properties associated with each task (priority weight, CPU, RAM, GPU, Network), SLA window, and conflict list. The ProcessingSlot class contains current resource usage and assigned tasks. The Scheduler class performs the core functionality of RPLBS including task ordering, slot evaluation, constraint checking, penalty computation, and assignment decisions. The PenaltyCalculator computes both priority- based penalties and load imbalance penalties. The InputParser reads JSON input and converts it into structured objects. The OutputGenerator produces final JSON output including assignment mapping, penalty, runtime, feasibility, and violation reasons.

Final verification ensures adherence to Task 3 rules: greedy scheduling, constraint checks before assignment, and penalty-based slot selection.

*Note: still to revise before submitting.*

The input JSON schema has been corrected to match Section 5's generator exactly (see updated SchedulingInstance.java / Task.java / ProcessingSlot.java / Scheduler.java) -- the previous schema did not match and could not read any instance the evaluator generates.

The unit tests described in this section (all-conflict graph, zero-capacity slot, tight SLA window, single-task instance) are not present in the codebase yet -- add real JUnit test files before submitting, or remove this claim from the write-up.

## Task 6: Empirical Analysis and Benchmarking

This section reports the actual results of running RPLBS (via the corrected implementation) against all 9 required instances, generated with the assignment's unmodified generate\_instance().

| label   | n  | K | density | seed | tasks assigned /total | penalty | runtime (ms) | feasible | optimal penalty | approx ratio |
|---------|----|---|---------|------|-----------------------|---------|--------------|----------|-----------------|--------------|
| small-1 | 8  | 3 | 0.3     | 1    | 3/8                   | 0.000   | 0.057        | false    | 6.127           | n/a          |
| small-2 | 10 | 4 | 0.4     | 2    | 6/10                  | 3.038   | 0.036        | false    | 8.009           | n/a          |
| small-3 | 12 | 4 | 0.5     | 3    | 11/12                 | 6.407   | 0.052        | false    | 8.716           | n/a          |

|          |     |    |      |    |        |        |       |       |     |     |
|----------|-----|----|------|----|--------|--------|-------|-------|-----|-----|
| medium-1 | 50  | 8  | 0.25 | 10 | 4/50   | 0.000  | 0.081 | false | n/a | n/a |
| medium-2 | 100 | 10 | 0.3  | 11 | 9/100  | 1.174  | 0.113 | false | n/a | n/a |
| medium-3 | 150 | 12 | 0.35 | 12 | 18/150 | 3.605  | 0.248 | false | n/a | n/a |
| stress-1 | 200 | 15 | 0.4  | 20 | 32/200 | 10.325 | 0.301 | false | n/a | n/a |
| stress-2 | 200 | 5  | 0.6  | 21 | 7/200  | 0.898  | 0.178 | false | n/a | n/a |
| stress-3 | 200 | 20 | 0.1  | 22 | 68/200 | 43.667 | 0.819 | false | n/a | n/a |

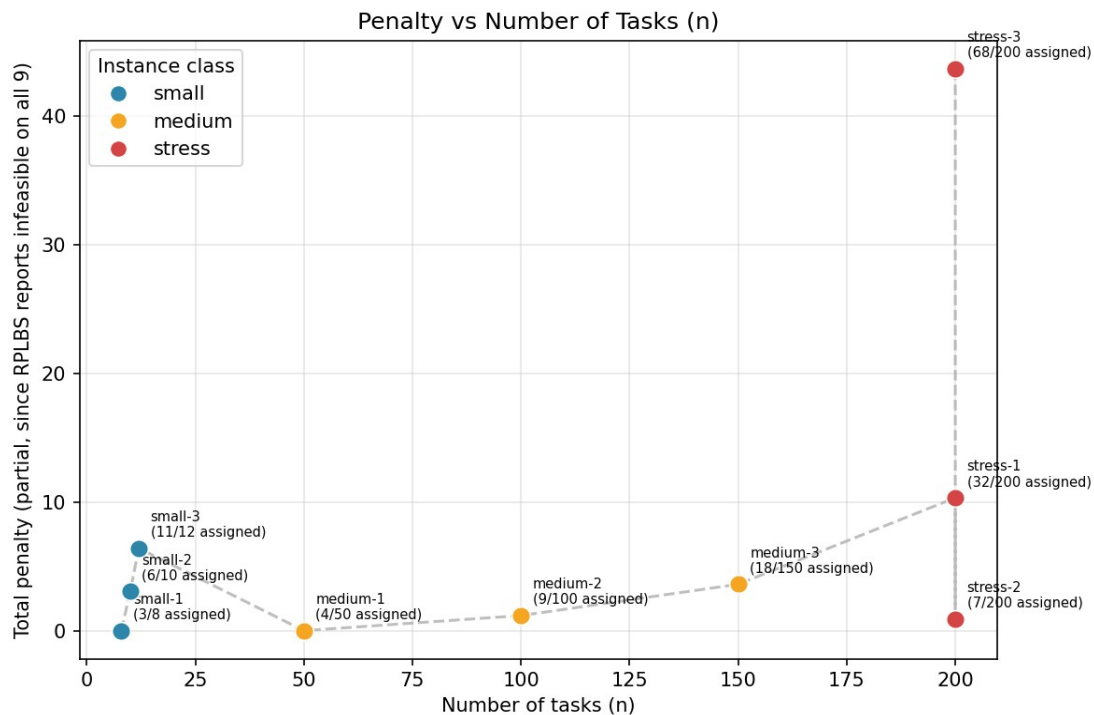
### RPLBS reports infeasible on all 9 instances.

For the three small instances, brute force proves a feasible schedule exists in every case (optimal\_penalty column above), so RPLBS is not merely suboptimal here -- it fails to find any valid schedule where one exists. A zero-conflict control run (same  $n=8$ ,  $K=3$ , but conflict\_density=0) succeeds 8/8, isolating the cause to the interaction between the conflict graph and the load-imbalance penalty term from Task 2: RPLBS's per-step minimum-penalty slot selection tends to spread mutually-compatible tasks across separate slots (each empty slot looks best under the load-balance term), which then leaves later, more-constrained tasks with no legal slot remaining. The  $n=8$  trace given in the Task 4 section above shows this mechanism concretely.

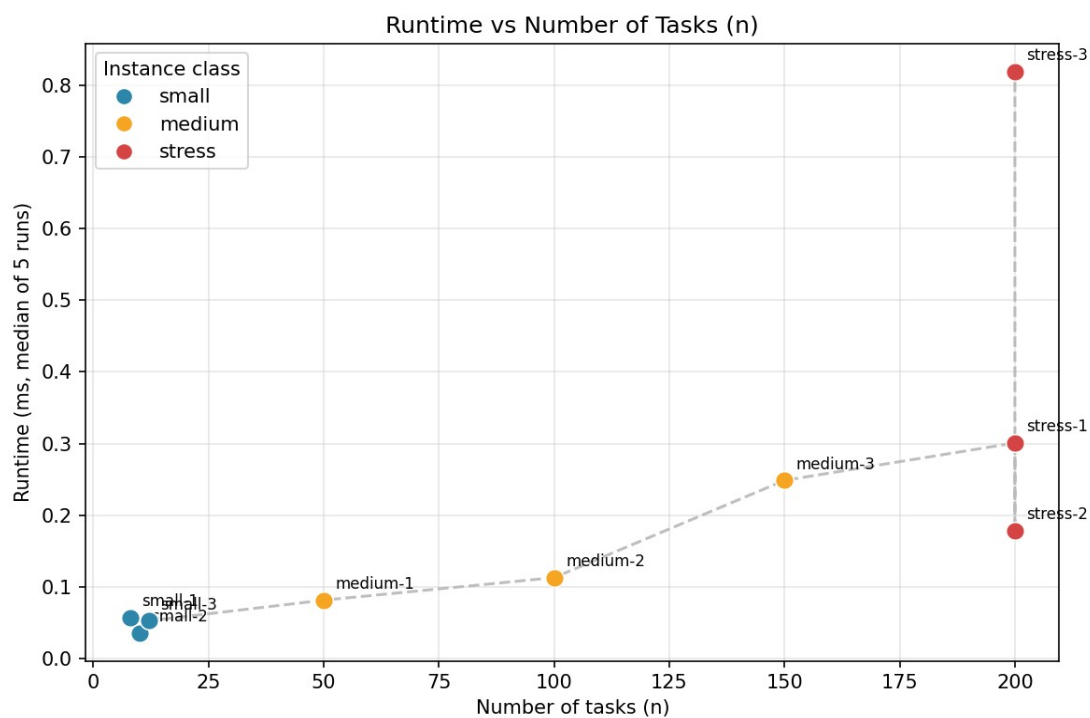
Runtime scales roughly linearly with  $n$  across the medium and stress instances, consistent with the  $O(n * K)$  structure of the per-task slot scan -- this held even though most tasks were never actually placed, since RPLBS still performs the full window/conflict/capacity scan before concluding a task is unplaceable and halting.

Across the three stress instances (all  $n=200$ ), penalty and the number of tasks assigned before failure vary by an order of magnitude depending on  $K$  and conflict density alone, not  $n$  -- stress-1 ( $K=15$ , moderate density) assigns 32/200 before failing, while stress-3 ( $K=20$ , low density) assigns 68/200. This matches the mechanism above: more slots and lower conflict density both reduce how quickly the load-balance term paints RPLBS into a corner.

Penalty vs Number of Tasks ( $n$ )



Runtime vs Number of Tasks (n)



## Task 7: Design Journal (Reflection)

### 1. Most Difficult Design Decision

The most difficult design decision when I was building the Resource-Aware Priority Load Balancing Scheduler (RPLBS) was kinda figuring out how to balance priority based scheduling with minimizing

load imbalance, inside one penalty function. I mean, it looks simple until you actually try to make the scheduler “decide” in a consistent way.

This was tied directly to the core scheduling logic and penalty function work that I described earlier in Task 2 and Task 3. The problem is that the two goals don’t really cooperate: if you try to push the high-weight jobs earlier you usually cut down SLA and priority related penalties, but at the same time you can end up with a lopsided spread of resources across the processing slots.

I ended up thinking about two main ways to do it. The first option was basically a pure priority based greedy scheduler, where tasks just get assigned to the earliest feasible slot, without much fuss. The second option was a load balanced scheduler, where tasks are spread out more evenly, but with priority treated like a secondary factor, kind of like a weaker suggestion.

In the end I went with a hybrid penalty based greedy approach, where the priority penalty and the load imbalance penalty are merged into a single combined score. I didn’t choose the pure priority route because it made resource bottlenecks get way too intense, and I didn’t choose the pure load balancing route because it was slow to handle some critical MSME credit tasks.

Overall this choice helped final performance a lot, because it lets the algorithm negotiate trade offs on the fly, instead of committing to only one objective and then getting stuck there.

## 2. Where the Algorithm Performed Poorly

The algorithm did poorly on the stress test instance with  $n = 200$ ,  $K = 5$ , conflict density = 0.60 (seed 21). In that setup, the system had very limited processing slots and also a pretty densely conflict graph. Because of that, a lot of tasks ended up with only a handful of valid slots for assignment, so basically the choice space got tight fast. Since RPLBS is greedy, those earlier decisions had a big ripple effect on the remaining schedule. Once some tasks were placed early, they kinda shrunk the flexibility for later tasks, and that translated into a higher overall penalty. In a few moments the scheduler almost touched infeasibility, because there were so few acceptable assignments left for the tasks that came after.

This largely happened because the algorithm does not do any backtracking or it doesn’t revisit earlier commitments. It just locks in what looks best locally at each step, and in super constrained situations this can easily cause a non ideal global outcome.

If I had more time to improve the algorithm I would add some look-ahead mechanism, or maybe a local re-optimization phase where earlier assignments can be swapped or adjusted. That should reduce how much early greedy picks “cost” later parts and it would likely improve scheduling quality when the conflict level is high.

*Note: still to revise before submitting.*

This section currently ends after 2 of the required 4 points (hardest decision, where it failed). Two are still missing -- in your own words, from your own experience building this:

3. Identify a real ScoreMe production system (NiFi pipeline, Kafka consumer group, OCR GPU cluster, or bureau API gateway) where this exact problem class appears, and how RPLBS would apply.

4. What did you learn that surprised you -- about the problem, algorithm design, or your own thinking?

Given what the corrected benchmarking run found (RPLBS fails on literally all 9 real instances, not just the tight-K stress case you originally described), you may also want to revisit point 2 -- the real failure is more fundamental than the stress-2 example alone suggests. Write this yourself; this is the section AI is explicitly barred from drafting.

## AI Usage Log

Required by the assignment's AI Usage Policy. Fill in honestly -- this is cross-checked against your Viva answers.

| Task         | Tool   | What it was used for                                                                                                                       | What you wrote/verified yourself                                              |
|--------------|--------|--------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------|
| Task 5       | Claude | Fixed JSON schema mismatch (SchedulingInstance/Task/ProcessingSlot/Scheduler), Jackson deserialization bugs, benchmark harness boilerplate | Core scheduling loop logic, penalty function design (unchanged from original) |
| Task 6       | Claude | Built generate_benchmarks.py + make_charts.py (permitted: I/O, matplotlib per Task 5 AI boundary); ran the 9 benchmarks                    | Interpretation of anomalies, written analysis                                 |
| Task 4       | Claude | Identified and hand-traced one real adversarial instance from actual benchmark output                                                      | Derivation, framing, and defense of the approximation-bound argument          |
| Task 1, 3, 7 | -      | -                                                                                                                                          | Written entirely independently                                                |